

CSCI235 – Database System

PL/pgSQL

Sionggo Japit

sjapit@uow.edu.au

29 March 2026

Outline

What is PL/pgSQL?

Why do we need it?

Program structure

Declarative and Executable components

Functions and procedures

Data types, implicit type declarations

Operators

Control statements

Cursors

Exceptions

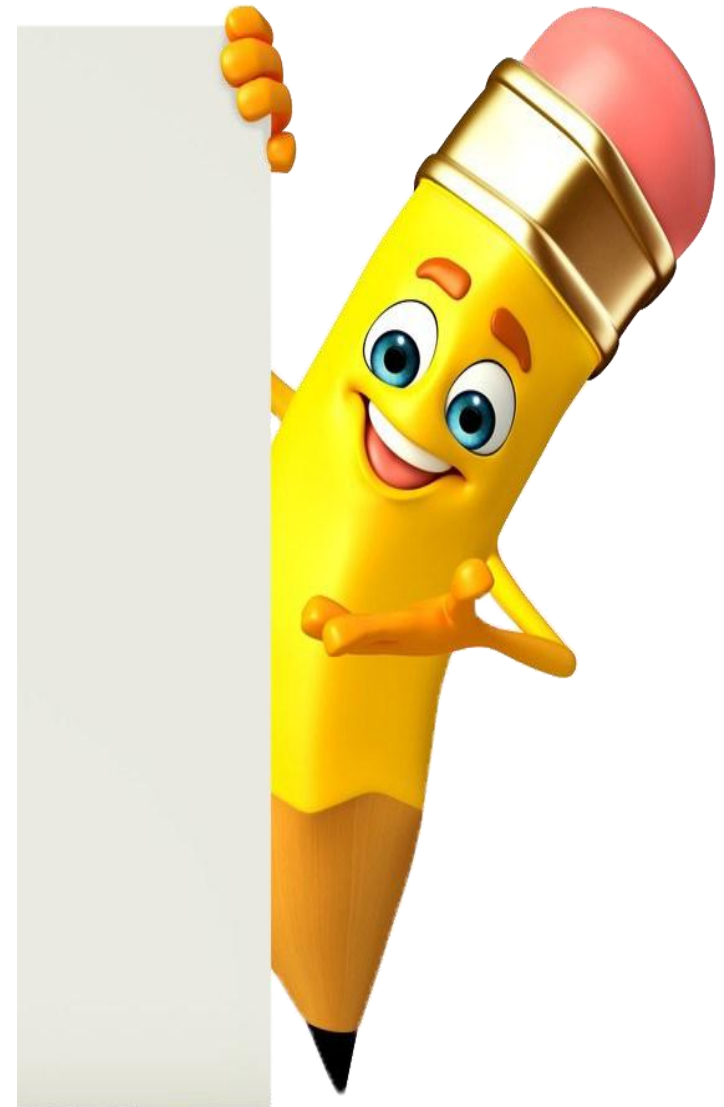
Anonymous code blocks



- PL/pgSQL is a procedural extension of SQL.
- PL/pgSQL = procedural **P**rogramming **L**anguage + **pgSQL** (PostgreSQL)
- We need PL/pgSQL to bridge a gap between a high-level declarative query language and a procedural programming language
- PL/pgSQL =
 - Data Manipulation statements of SQL +
 - SELECT statement +
 - Variables +
 - Assignment statement +
 - Conditional control statements +
 - Repetition statement +
 - Exception handling +
 - Procedure and function statements.

PL/pgSQL

Program Structure



Program Structure

- PL/pgSQL is a block-structured language
- It means that the basic units of PL/pgSQL programs such as functions and procedures have the structures of nested blocks.
- A block consists of the optional label and declarative components and a compulsory executable component

```
[ <<Label>> ]  
[ DECLARE  
    declarations ]  
BEGIN  
    statements  
END [ Label ];
```

Program Structure

- Blocks can be nested to any level
- A declarative component consists of declarations of constants, variables, types, methods, cursors, etc., and it is optional.
- An executable component consists of executable code.

Program Structure

- A label is needed to identify the block when EXIT statement is used, or to qualify the names of the variables declared in the block.
- Any statement in an executable component of a block can be a nested block.

```
CREATE FUNCTION nested() RETURN integer AS $$  
DECLARE  
    value integer := 5;  
BEGIN  
    DECLARE  
        value integer := 10;  
    BEGIN  
        RAISE NOTICE 'Inner value= %', value; -- 10  
    END;  
    RAISE NOTICE 'Outer value= %', value; -- 5  
    RETURN value;  
END;  
$$ LANGUAGE plpgsql;
```

Nested blocks



PL/pgSQL

Declarative and
Executable Components

Declarative Components

Declarative components contain declarations of variables, constants and cursors

<code>DECLARE</code>	Start of declarations
<code>value INTEGER;</code>	Integer variable, 4 bytes
<code>stock_num DECIMAL(5);</code>	Binary coded decimal
<code>stock_num NUMERIC(5);</code>	Equivalent to DECIMAL
<code>stock_name CHAR(5);</code>	fixed size string
<code>stock_name VARCHAR(30);</code>	Variable size string, unlimited length
<code>stock_date DATE;</code>	Date type
<code>stock_required CONSTANT NUMERIC(5) := 30;</code>	Numerical constant
<code>max_salary CONSTANT NUMERIC(11,2) := 2.45;</code>	Numerical Constant
<code>course_row COURSE%ROWTYPE;</code>	Type derived from a row
<code>course_number COURSE.cnum%TYPE;</code>	Type derived from a column
<code>query CURSOR FOR SELECT * FROM COURSE;</code>	Cursor

Executable Components

Executable components include assignment statements, conditional control statements, iterative statements, procedure and function calls, SQL statements

```
department_code := 'SCIT';
```

Assignment statement

```
SELECT name  
INTO STRICT department_name  
FROM DEPARTMENT  
WHERE code = department_code;
```

SELECT statement with assignment

```
IF (credits < limit) THEN  
    credits := credits + 1;  
    limit := limit - 1;  
ELSIF (credits = limit) THEN  
    credits := credits - 2;  
ELSE  
    limit := limit + 1;  
END IF;
```

IF statement

```
FOR i IN 1..100 LOOP  
    total := total + i;  
END LOOP;
```

FOR statement

Exception Component

Exception component consists of executable statements that service the exceptional situations during execution

```
EXCEPTION
```

No results from SELECT statement

```
WHEN NO_DATA_FOUND THEN
```

```
    INSERT INTO AUDIT_TABLE VALUES( current_timestamp, course_number );
```

```
    RAISE NOTICE 'Course % not found', course_number;
```

```
EXCEPTION
```

Two many rows for a single variable

```
WHEN TOO_MANY_ROWS THEN
```

```
    RAISE EXCEPTION 'Course % not unique', title;
```

```
EXCEPTION
```

Division by zero

```
WHEN DIVISION_BY_ZERO THEN
```

```
    RAISE NOTICE 'Caught division_by_zero';
```

```
EXCEPTION
```

Any possible exception

```
WHEN OTHERS
```

```
    INSERT INTO AUDIT_TABLE VALUES( current_timestamp, 'Error' );
```

```
    RAISE EXCEPTION 'Something went wrong';
```

```
END;
```

PL/pgSQL
Function and
Procedure



Functions

A birds-eye view of a **function** is the following

<code>CREATE OR REPLACE FUNCTION function_name (parameters)</code>	Statement code
<code>RETURN type-specification AS</code>	Type of value retruned
<code>\$\$ -- Function body separator</code>	Start separator
<code>DECLARE</code>	Keyword
<code>-- optional declarations</code>	Declarations
<code>BEGIN</code>	Keyword
<code>-- executable statements, at least one statements is required</code>	Statements
<code>RETURN value;</code>	Value returned
<code>EXCEPTION</code>	Keyword
<code>-- optional exception handlers</code>	Exceptions
<code>END;</code>	Keyword
<code>\$\$ -- Function body separator</code>	End separator
<code>LANGUAGE plpgsql;</code>	Langage used

Functions

A sample **function** `hello_world`

Fuction `hello_world`

```
CREATE OR REPLACE FUNCTION hello_world( hello text,  
                                         world text )  
  
RETURN text AS  
$$  
DECLARE  
    hello_message text;  
BEGIN  
    hello_message := hello || ' ' || world || ' !';  
    RETURN hello_message;  
END;  
$$  
LANGUAGE plpgsql;
```

A sample processing of `hello_world` **function**

Processing of `hello_world` function

```
SELECT hello_world('Hello', 'world');  
hello_world  
-----  
Hello world !
```

Functions

Processing of SQL statements within a `function raise_budget`

<code>CREATE OR REPLACE FUNCTION raise_budget(department_name text,</code>	Function name and paratemeters
<code>budget_limit NUMERIC)</code>	Parameters
<code>RETURN NUMERIC AS \$\$</code>	Type of value returned and start separator
<code>DECLARE</code>	Keyword
<code>current_budget DEPARTMENT.budget%TYPE;</code>	Declaration of variable
<code>BEGIN</code>	Keyword
<code>SELECT budget INTO current_budget FROM DEPARTMENT WHERE name = department_name;</code>	SELECT statement
<code>IF current_budget < budget_limit THEN</code>	IF statement
<code>UPDATE DEPARTMENT SET budget = budget_limit WHERE name = department_name;</code>	UPDATE statement
<code>RETURN budget_limit;</code>	Value returned
<code>ELSE</code>	ELSE clause
<code>INSERT INTO AUDIT VALUES('Budget OK', current_budget);</code>	INSERT statement
<code>RETURN current_budget;</code>	Value returned
<code>END IF;</code>	END IF keyword
<code>END; \$\$</code>	END keyword and end separator
<code>LANGUAGE plpgsql;</code>	Language used

Procedures

A birds-eye view of a **procedure** is the following

<code>CREATE OR REPLACE PROCEDURE procedure_name (parameters) AS</code>	Procedure name and parameters
<code>\$\$ -- Procedure body separator</code>	Start separator
<code>DECLARE</code>	Keyword
<code>-- optional declarations</code>	Declarations
<code>BEGIN</code>	Keyword
<code>-- executable statements, at least one statements is required</code>	Executables
<code>EXCEPTION</code>	Keyword
<code>-- optional exception handlers</code>	Exceptions
<code>END;</code>	Keyword
<code>\$\$</code>	End separator
<code>LANGUAGE plpgsql;</code>	Language used

Procedures

A sample `phello_world` procedure

Procedure `phello_world`

```
CREATE OR REPLACE PROCEDURE phello_world ( hello text,  
                                           world text,  
                                           OUT message text) AS  
  
$$  
BEGIN  
    message := hello || ' ' || world || ' !';  
END;  
$$  
LANGUAGE plpgsql;
```

A `CALL` statement can be used to process a `procedure phello_world`

Processing of `phello_world` procedure

```
CALL phello_world('Hello', 'world', NULL);  
    message  
-----  
    Hello world !  
(1 row)
```

Procedures

Processing SQL statements within a **procedure** `praise_budget`

<code>CREATE OR REPLACE PROCEDURE praise_budget(department_name text, budget_limit NUMERIC) AS</code>	Procedure
<code>\$\$</code>	Start separator
<code>DECLARE</code>	Keyword
<code>current_budget DEPARTMENT.budget%TYPE;</code>	Declaration
<code>BEGIN</code>	Keyword
<code>SELECT budget INTO current_budget FROM DEPARTMENT WHERE name = department_name;</code>	SELECT statement
<code>IF current_budget < budget_limit THEN</code>	IF statement
<code>UPDATE DEPARTMENT SET budget = budget_limit WHERE name = department_name;</code>	UPDATE statement
<code>ELSE</code>	Keyword
<code>INSERT INTO AUDIT VALUES('Budget OK', current_budget);</code>	INSERT statement
<code>END IF;</code>	Keyword
<code>END;</code>	Keyword
<code>\$\$</code>	End separator
<code>LANGUAGE plpgsql;</code>	Language used



PL/pgSQL

Data types, implicit
type declarations

PL/pgSQL Operators



Operators

Arithmetic operators

`+, -, *, /, ^`

Arithmetic

Relational operators

`<, >, >=, <=, =, !=, <>`

Relational

Comparison operators

`LIKE, BETWEEN, IN, IS NULL, =, !=, <>`

Comparison

Boolean operators

`AND, OR, NOT`

Boolean

String operator

`||`

String

Operator precedence

`(unary +, -), (^), (*, /), (+, -, ||), (comparison), (NOT), (AND), (OR)`

Precedence



PL/pgSQL

Control Statements

Conditional Control Statements

A birds-eye view of **conditional control statements** is the following

```
IF condition THEN
    statement;
    ...
ELSE
    statement;
    ...
END IF;
```

IF statement

```
IF condition THEN
    statement;
    ...
ELSIF condition THEN
    statement;
    ...
ELSIF condition THEN
    statement;
    ...
ELSE
    statement;
    ...
END IF;
```

IF statement

Iterative Control Statements

A birds-eye view of **iterative control statements** is the following

LOOP

LOOP statement

statement;

...

IF condition THEN EXIT;

statement;

...

END IF;

statement;

...

END LOOP;

FOR variable IN scope

FOR statement with LOOP statement

LOOP

statement;

...

END LOOP;

FOR variable IN REVERSE scope

FOR statement with LOOP statement

LOOP

statement;

...

END LOOP;

Iterative Control Statements

A birds-eye view of **iterative control statements** is the following

```
WHILE (condition)
```

WHILE loop

```
LOOP
```

```
    statement;
```

```
    ...
```

```
END LOOP;
```

```
LOOP
```

EXIT a loop

```
    statement;
```

```
    ...
```

```
    EXIT WHEN condition;
```

```
    statement;
```

```
    ...
```

```
END LOOP;
```

```
<<label>> BEGIN
```

EXIT a block

```
    statement;
```

```
    ...
```

```
    IF condition THEN EXIT label;
```

```
    statement;
```

```
    ...
```

```
END;
```

PL/pgSQL

Cursors



Cursors

What happens when `SELECT` statement returns more than one row ?

```
CREATE OR REPLACE PROCEDURE error() AS $$  
DECLARE  
    course_title COURSE.title%TYPE;  
BEGIN  
    SELECT title INTO STRICT course_title FROM COURSE WHERE cnum IN ('CSCI235','CSCI205');  
    RAISE NOTICE 'Title: %',course_title;  
END;  
$$ LANGUAGE plpgsql;
```

Stored procedure

```
call error();  
ERROR:  query returned more than one row  
HINT:  Make sure the query returns a single row, or use LIMIT 1.  
CONTEXT:  PL/pgSQL function error() line 5 at SQL statement
```

Processing error

A variable `course_title` with a keyword `STRICT` cannot be used to store several rows retrieved from a relational table

A solution is to process the rows in a `row by row` mode

A **cursor** is a construction that allows for processing the rows retrieved from the relational tables in a `row by row` mode

Cursors

Declaration and processing of a **bound cursor**

```
CREATE OR REPLACE PROCEDURE BCURSOR() AS
```

Declaration of a bound cursor variable

```
$$
```

```
DECLARE
```

```
  TITLES CURSOR FOR SELECT title FROM COURSE WHERE cnum IN ('CSCI235','CSCI205');
```

```
  course_title text;
```

```
BEGIN
```

Processing of a bound cursor

```
  OPEN TITLES;
```

```
  LOOP
```

```
    FETCH TITLES INTO course_title;
```

```
    EXIT WHEN NOT FOUND;
```

```
    RAISE NOTICE 'Title: %',course_title;
```

```
  END LOOP;
```

```
  CLOSE TITLES;
```

```
END;
```

```
$$
```

```
LANGUAGE plpgsql
```

Processing of a stored procedure with a cursor

```
CALL BCURSOR();
```

Cursors

Declaration and processing of an **unbound cursor**

```
CREATE OR REPLACE PROCEDURE UCURSOR() AS
```

Declaration of an unbound cursor variable

```
$$
```

```
DECLARE
```

```
    course_title text;
```

```
    TITLES REFCURSOR;
```

```
BEGIN
```

Processing of an unbound cursor

```
    OPEN TITLES FOR SELECT title FROM COURSE WHERE cnum IN ('CSCI235','CSCI205');
```

```
    LOOP
```

```
        FETCH TITLES INTO course_title;
```

```
        EXIT WHEN NOT FOUND;
```

```
        RAISE NOTICE 'Title: %',course_title;
```

```
    END LOOP;
```

```
    CLOSE TITLES;
```

```
END;
```

```
$$
```

```
LANGUAGE plpgsql;
```

Processing of a stored procedure with a cursor

```
CALL UCURSOR();
```

Cursors

Declaration and implicit processing of a **bound cursor**

Declaration of a bound cursor variable

```
CREATE OR REPLACE PROCEDURE IBCURSOR() AS
$$
DECLARE
    TITLES CURSOR FOR SELECT title FROM COURSE WHERE cnum IN ('CSCI235','CSCI205');
```

Implicit processing of a bound cursor

```
BEGIN
    FOR trow IN TITLES
    LOOP
        RAISE NOTICE 'Title: %',trow.title;
    END LOOP;
END;
$$
LANGUAGE plpgsql;
```

Processing of a stored procedure with a cursor

```
CALL IBCURSOR();
```

PL/pgSQL Exceptions



Exceptions

An **exception** is an internally defined or user defined error condition, e.g. divide by zero, no rows selected by **SELECT** statement with **INTO** clause, failure of **FETCH** statement, use of a cursor which has not been opened yet, etc.

Handling of **no rows selected exception**

```
CREATE OR REPLACE PROCEDURE exception1() AS
$$
DECLARE
    course_title COURSE.title%TYPE;
BEGIN
    SELECT title INTO STRICT course_title FROM COURSE WHERE cnum = 'ABCD123';
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE NOTICE 'Course ABCD123 not found';
END;
$$
LANGUAGE plpgsql;
```

No rows selected

```
call exception1();
NOTICE: Course ABCD123 not found
```

No rows selected

Exceptions

Handling too many rows selected exception

```
CREATE OR REPLACE PROCEDURE exception2() AS
$$
DECLARE
    course_title COURSE.title%TYPE;
BEGIN
    SELECT title INTO STRICT course_title FROM COURSE;
EXCEPTION
    WHEN TOO_MANY_ROWS THEN
        RAISE NOTICE 'Too many rows found';
END;
$$
LANGUAGE plpgsql;
```

Too many rows selected

```
call exception2();
NOTICE: Too many rows found
```

Too many rows selected

Exceptions

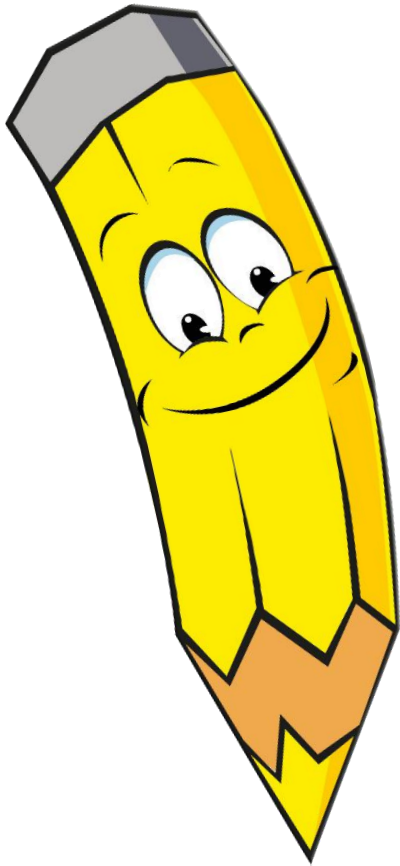
Handling of **other exception** together with a diagnostics report

```
CREATE OR REPLACE PROCEDURE exception3() AS $$
DECLARE
    message text;
    stack text;
    i integer := 5;
BEGIN
    i := i/0.0;
EXCEPTION
    WHEN OTHERS THEN
        GET STACKED DIAGNOSTICS message := MESSAGE_TEXT,
                                stack := PG_CONTEXT;
        RAISE NOTICE 'Message text: %', message;
        RAISE NOTICE E'--- Call Stack ---\n%', stack;
END; $$
LANGUAGE plpgsql;
```

Other

```
call exception3();
NOTICE:  Message text: division by zero
NOTICE:  --- Call Stack ---
PL/pgSQL function exception3() line 10 at GET STACKED DIAGNOSTICS
```

Other



PL/pgSQL

Anonymous Code
Blocks

Anonymous Code Block

Anonymous code block is a **transient anonymous function** in a procedural language

Syntactically **anonymous code block** is a block of code without **CREATE FUNCTION** or **CREATE PROCEDURE** header

A statement **DO** can be used to process **anonymous code block**

A sample **anonymous code block**

```
DO
$$
BEGIN
    IF (SELECT count(*) FROM COURSE) > 0 THEN
        RAISE NOTICE 'COURSE table is not empty';
    END IF;
END;
$$
LANGUAGE plpgsql;
```

Anonymous code block

Anonymous Code Block

Yet another sample **anonymous code block**

Anonymous code block

```
DO
$$
DECLARE
    dept_name DEPARTMENT.name%TYPE := 'Computer Science';
    dept_code DEPARTMENT.code%TYPE;
BEGIN
    SELECT name, code
    INTO dept_name, dept_code
    FROM DEPARTMENT
    WHERE name = dept_name;

    RAISE NOTICE 'A code of department % is %', dept_name, dept_code;
END;
$$
LANGUAGE plpgsql;
```

References

[PostgreSQL 16.1 Documentation, Chapter 43. PL/pgSQL — SQL Procedural Language](#)

[PostgreSQL Tutorial, PostgreSQL PL/pgSQL](#)

T. Connolly, C. Begg, Database Systems, A Practical Approach to Design, Implementation, and Management, Chapter 8 Advanced SQL, Pearson Education Ltd, 2015